React + TypeScript — Dalison's Cheat Sheet

A personal reference built from our lessons. It leans on the exact things you asked about and the spots that were tricky. Covers Lessons 1–9 (core).

⭐ The golden rules (the ones you asked about most)

1. In `onClick`, put a *function*, never a *function call*

This one came up three times — lock it in:

```
onClick={handleClick}          // ✅ no arguments → pass the reference
onClick={() => doThing(arg)}   // ✅ need to pass an argument → wrap in an arrow
onClick={doThing(arg)}         // ❌ the () runs it NOW, during render
```

- `onClick` wants a function to run *later* (on click).
- Adding `()` calls it **immediately, while rendering** — wrong, and often a bug/loop.
- **No argument?** pass the name alone: `onClick={onLike}`.
- **Need to pass an argument?** wrap it: `onClick={() => navigate(-1)}`.
- The TS error "*void is not assignable to MouseEventHandler*" means exactly this: you passed the **result** of calling the function (nothing), not the function.

2. State: `const`, only the setter changes it, functional updater when it builds on itself

```
const [count, setCount] = useState(0);
setCount(count + 1);          // fine for a one-off
setCount(prev => prev + 1);   // use this when the new value depends on the old
```

- It's `const` because within one render the value never changes — it's a frozen snapshot. A new render makes a fresh one.
- **Never** reassign the variable (`count = 5`) — nothing happens. Only the setter triggers a re-render.
- Use `prev => ...` when the next value is based on the current one. It reads the freshest value, avoiding a stale snapshot (and, with objects, avoiding a "lost update" that erases sibling data).

3. `className`, not `class` — and fix console warnings

- JSX uses `className` (and `htmlFor`, not `for`).
- Some mistakes only *warn* in the console instead of breaking. Treat a warning as "not done." A clean console is the standard on real teams.

4. TypeScript types vanish at runtime

- Types are checked **in the editor and by** `npm run build` — never while the app runs. The dev server strips them and runs plain JavaScript.
- So a red squiggle can still "work" in the browser. `npm run type-check` / `npm run build` are the gate that actually enforces types.

Components & JSX (Lesson 1)

```tsx
export default function Hero() {
  const name = "Dalison";
  return <h1 className="hero-title">{name}</h1>;
}
```

- A component is a function that returns markup. **Capitalized** name = your component; lowercase = real HTML tag.
- Must return **one** root element — wrap siblings in `<>...</>` (a Fragment).
- `{ }` = "drop a JavaScript value in here." Works for any expression: `{name}`, `{2 + 2}`, `{items.map(...)}`.

Props (Lesson 2)

```tsx
// parent
<ProjectCard project={project} />

// child
interface ProjectCardProps { project: Project; }
export default function ProjectCard({ project }: ProjectCardProps) { ... }
```

- Props = a component's inputs. Data flows **down** (parent → child) and is **read-only**.
- Conditional rendering: `{condition && <Thing />}` → renders `<Thing />` only if `condition` is true.

State — `useState` (Lesson 3)

```tsx
const [open, setOpen] = useState(false);   // boolean inferred
const [likes, setLikes] = useState<Record<string, number>>({});  // explicit type
```

- State = data that changes over time and belongs to a component. Changing it re-renders.
- **Per instance**: each rendered copy of a component has its own state.
- See golden rule #2 for the `const` / setter / functional-updater details.

Effects — `useEffect` (Lesson 4)

```tsx
useEffect(() => {
  const onScroll = () => setVisible(window.scrollY > 400);
  window.addEventListener("scroll", onScroll);   // setup
  return () => window.removeEventListener("scroll", onScroll); // cleanup
}, []); // dependency array
```

- For **side effects** — things outside React: listeners, timers, the DOM, fetching.
- **Dependency array** controls *when the setup re-runs*:
  - no array → after every render
  - `[]` → once, on mount
  - `[x]` → on mount, then whenever `x` changes
- **Cleanup** = the returned function. Runs before the effect re-runs and when the component is removed. Rule: whatever you *start*, you *stop*.
- In dev (`StrictMode`) effects run **twice** on mount on purpose — it's testing your cleanup, not a bug.

Lifting state up (Lesson 5)

When siblings need to share state, move it to their nearest common parent.

- **Data down, events up**: parent passes the value + a callback down; child calls the callback to report an event; the parent owns the state.
- **Controlled component**: a child that owns no state — just gets a value and a callback (e.g. `ProjectCard` after the refactor).
- **Derived state**: compute values (like a total) from state; don't store what you can calculate.

Immutable object update (you asked for the full breakdown):

```tsx
setLikes(prev => ({ ...prev, [id]: (prev[id] ?? 0) + 1 }));
```

- `(prev) => ({ ... })` — the parens make the `{ }` an **object to return**, not a function body.
- `...prev` — copy every existing key into a **new** object (never mutate `prev`).
- `[id]:` — a **computed key**: use the value of the variable `id` as the key.
- `?? 0` — if the left side is `null` / `undefined`, use `0` instead.
- Last key wins, so `[id]:` **overrides** just that one field after the spread.

Forms & controlled inputs (Lesson 6)

```
const [form, setForm] = useState({ name: "", email: "", message: "" });

const handleChange = (e: React.ChangeEvent<HTMLInputElement |
HTMLTextAreaElement>) => {
  const { name, value } = e.target;
  setForm(prev => ({ ...prev, [name]: value })); // same immutable pattern
};

<input name="name" value={form.name} onChange={handleChange} />
```

- **Controlled input** = `value` comes *from* state, `onChange` writes *back* to state. React is the single source of truth. Letters appear because state updated and re-rendered — not directly from typing.
- One state object + one handler that uses the input's `name` = updates any field.
- `<form onSubmit={...}>` + `e.preventDefault()` stops the browser's full-page reload so you handle it in JS.
- **Controlled vs uncontrolled** (the experiment): with `value` bound, React can set/clear/format the field. Remove `value` and the DOM owns it — you can no longer clear it from code. **Prefer controlled** (`value` + `onChange` together).

Context — `useContext` (Lesson 7)

Purpose: share a value app-wide **without passing props through every layer** (prop drilling).

```
const ThemeContext = createContext<ThemeContextValue | undefined>(undefined);

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<"light" | "dark">("dark");
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
```

```
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (context === undefined) throw new Error("useTheme must be inside
ThemeProvider");
  return context;
}
```

- `createContext` returns an **object**; its `.Provider` is the component.
- `value={{...}}` on the Provider is exactly what every `useContext` reader receives. Double braces = "JSX expression" (outer) + "object literal" (inner).
- The generic on `createContext<T>` types both the `value` prop **and** what `useContext` returns.
- The `undefined` guard turns a confusing crash into a clear error message.

Routing — react-router (Lesson 8)

```
// main.tsx: wrap the app
<BrowserRouter> <App /> </BrowserRouter>

// App.tsx: map URLs to components (Navbar/Footer stay outside = on every
page)
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/projects/:id" element={<ProjectDetail />} />
  <Route path="*" element={<Home />} />   {/* fallback */}
</Routes>

// navigating
<Link to={`/projects/${id}`}>View</Link>   // no page reload
const { id } = useParams();                 // read the :id from the URL
const navigate = useNavigate();             // navigate(-1) = go back
```

- React apps are single-page — the router **fakes** navigation by swapping components and updating the URL, with no full reload.
- `:id` is a dynamic segment; `useParams()` reads it (typed `string | undefined`, so guard it).
- `<Link>` vs `<a>`: `Link` = instant in-app; `<a>` = full reload (loses state).

Data fetching (Lesson 9)

```
const [repos, setRepos] = useState<Repo[]>([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  let ignore = false;                        // per-run flag; cleanup flips it

  async function load() {                    // can't make the effect itself
async
    setLoading(true);
    setError(null);
    try {
      const res = await fetch(url);          // 1st await: response arrives
      if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch WON'T do
this
      const data: Repo[] = await res.json();// 2nd await: body read + parsed
      if (!ignore) setRepos(data);
    } catch (err) {
      if (!ignore) setError(err instanceof Error ? err.message : "Failed");
    } finally {
      if (!ignore) setLoading(false);        // always ends, success or fail
    }
  }

  load();                                    // fire-and-forget
  return () => { ignore = true; };           // cleanup: mark this run stale
}, []);
```

- **The four states**: loading, error, empty (`repos.length === 0`), success. Real UIs handle all four — render one block per state.
- **Two awaits** = two async stages: (1) `fetch` resolves when the response/headers arrive; (2) `res.json()` resolves when the body is fully read and parsed.
- `fetch` **does NOT reject on 404/500** — only on network failure. Check `res.ok` (true for 200–299) yourself and `throw` if it's false.
- `try / catch / finally` : risky work in `try` ; failures → `catch` ; `finally` always runs (put `setLoading(false)` there so you can never get stuck loading).
- **The** `ignore` **flag** stops an outdated fetch run from applying ANY result (success, error, or loading). Cleanup sets `ignore = true` when the component unmounts or the effect re-runs; each setter is guarded by `if (!ignore)` . Also makes the StrictMode dev double-fetch harmless.
- **Fire-and-forget**: calling `load()` without storing its promise is fine — we call it for side effects, and it handles its own errors inside.

Server state — TanStack Query (Lesson 10)

The library that replaces all the manual fetch boilerplate above (~40 lines → ~5).

```
// main.tsx — provide the client once, near the top
const queryClient = new QueryClient();
<QueryClientProvider client={queryClient}> ...app... </QueryClientProvider>

// in a component
async function fetchRepos(user: string): Promise<Repo[]> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

const { data: repos, isPending, isError, error } = useQuery({
  queryKey: ["repos", USERNAME],       // cache id
  queryFn: () => fetchRepos(USERNAME), // how to fetch (returns data or
throws)
});
```

- `queryFn` = pure fetch logic (no React); returns data or throws.
- `queryKey` = a unique cache label (array). Used to: cache the result, dedupe identical requests, and **refetch automatically when the key changes** (e.g. a different username). Two components with the *same key* share one result.
- Returns `isPending` / `isError` / `error` / `data` — no `useState` / `useEffect` / `try/catch` / `ignore` needed. Caching, retries, background refetch, and race handling come free.
- `data: repos` is destructure-and-rename (your choice of variable name). It is NOT related to the string `"repos"` in the queryKey — one is a variable name, the other is an arbitrary cache label (quotes = string/data; no quotes = identifier).
- TS: `data` is inferred as `Repo[] | undefined`; the `isPending` / `isError` guards narrow it to `Repo[]` in the success branch.

useReducer & state patterns (Lesson 11)

For state that's too complex for scattered `useState`s. Components **dispatch actions** ("what happened"); one **reducer** decides the new state.

```
type Action =
  | { type: "increment" }
  | { type: "add"; amount: number };   // discriminated union
```

```ts
function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment": return { count: state.count + 1 };
    case "add":       return { count: state.count + action.amount };
    default:          return state;
  }
}

const [state, dispatch] = useReducer(reducer, { count: 0 });
dispatch({ type: "add", amount: 10 });
```

- Flow: `dispatch(action)` → `reducer(state, action)` → new state → re-render. The reducer is the ONLY place state changes.
- **When to use it** (the real lesson): only when several values change together by rules, or there are many kinds of update. A simple form/toggle should stay on `useState` — reaching for a reducer there is overkill.
- Same idea as `Array.reduce` (accumulator in → new accumulator out) and it's the mental model behind Redux.
- **Discriminated union** (`type` field): inside `switch (action.type)` , TS narrows `action` to the exact shape per case — `action.amount` exists only in `"add"` .
- `dispatch(action)` passes ONLY the action — React supplies the current state to the reducer for you (same idea as the `prev =>` functional updater). The action carries just "what happened" + any payload (e.g. `amount`).
- The reducer function name is your choice (convention: `somethingReducer`). You can have several `useReducer` s per component, but the idiom is ONE reducer per cohesive chunk of state (often a whole object with many fields).
- Single value? You don't need a state object — `useReducer(reducer, 0)` with a `number` is fine (and honestly a single value usually wants plain `useState`).

Performance (Lesson 12)

Re-render model — a component re-renders when: (1) its own state changes, (2) its parent re-renders (cascades to ALL children), or (3) a context it uses changes. A re-render = "re-run the function + diff"; it does NOT rebuild the DOM (React only touches the DOM where output actually changed). **Re-rendering is cheap** — usually leave it alone.

```ts
// caches a VALUE — recompute only when deps change
const total = useMemo(() => computeTotal(items), [items]);

// caches a FUNCTION — same reference until deps change
const handleClick = useCallback(() => doThing(id), [id]);

// skips a child's re-render if its props are unchanged (shallow compare)
export default memo(MyComponent);
```

- Dependency array = same idea as `useEffect`'s: "only redo when a dep changes."
- `useMemo` gives back the **value**; `useCallback` gives back the **function**. In fact `useCallback(fn, d)` === `useMemo(() => fn, d)`.
- **Gotcha:** inline objects/functions are a NEW reference each render, so passing `onClick={() => …}` to a `memo`'d child breaks the memo — it sees "prop changed." Fix with `useCallback`. That's why `memo` + `useCallback` travel together.
- ⭐ **When NOT to optimize (the real lesson):** these add memory + complexity and most components are already fast. Write plain code; only add memo/useMemo/ useCallback when you MEASURE a real problem (React DevTools Profiler). Premature memoization is a classic mistake. (React 19's Compiler auto-memoizes anyway.)

Testing — Vitest + React Testing Library (Lesson 13)

Tests = code that checks your code, so you can change things later with confidence. Run with `npm test` (watch mode; re-runs on save, `q` to quit).

```
import { describe, it, expect } from "vitest";
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import About from "./About";

describe("About", () => {
  it("shows extra text after clicking Read more", async () => {
    render(<About />);                                          // ARRANGE
    expect(screen.queryByText(/rebuilding this/i)).not.toBeInTheDocument();
    await userEvent.click(screen.getByRole("button", { name: /read more/i }));
// ACT
    expect(screen.getByText(/rebuilding this/i)).toBeInTheDocument(); //
ASSERT
  });
});
```

- **Golden rule: test behavior, not implementation** — find things the way a user would (by visible text / role), assert what the user sees. Such tests survive refactors; tests tied to internals break needlessly.
- **Shape:** Arrange (`render`) → Act (`userEvent.click`) → Assert (`expect`).
- `getByText` / `getByRole` **throw** if not found (use when it should exist). `queryByText` returns **null** (use to assert something is ABSENT).
- `userEvent.click(...)` is `await`ed (real interactions are async).
- Text matchers take a string (whole-text exact) or a **regex** `/words/i` (substring, case-insensitive) — regex is the forgiving, common choice.
- Setup: `vite.config.ts` gets a `test` block (`environment: "jsdom"` , `setupFiles`); setup file imports `@testing-library/jest-dom/vitest` for matchers like `toBeInTheDocument()` and stubs missing browser APIs (e.g. IntersectionObserver, which jsdom lacks).

Testing real components (full lesson):

```
// Forms — type + submit; find inputs by their LABEL
const user = userEvent.setup();
await user.type(screen.getByLabelText(/name/i), "Dalison");
await user.click(screen.getByRole("button", { name: /send/i }));

// Needs context? Wrap in the provider (else the useTheme guard throws)
render(<ThemeProvider><Navbar /></ThemeProvider>);

// Needs routing? MemoryRouter + set the starting URL
render(
  <MemoryRouter initialEntries={["/projects/project-two"]}>
    <Routes><Route path="/projects/:id" element={<ProjectDetail />} />
</Routes>
  </MemoryRouter>
);
```

- `getByLabelText` = find a field by its `<label>` (needs `htmlFor` + `id`; doubles as an accessibility check). `user.type(el, "text")` fires real onChange events.
- A test must supply whatever providers the component needs (context, router), same as `main.tsx` does for the app. Real projects wrap this in one `renderWithProviders(ui)` helper.
- Test BOTH branches (e.g. ProjectDetail: valid id → shows it; bad id → not found).

Custom hooks + Rules of Hooks (Lesson 14)

A custom hook = a function named `useSomething` that calls other hooks, to extract reusable stateful logic. No special API — just a function.

```
export function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {          // lazy init: runs
once
    const stored = localStorage.getItem(key);
    return stored !== null ? (JSON.parse(stored) as T) : initialValue;
  });
  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));    // save on change
  }, [key, value]);
  return [value, setValue] as const;                     // same shape as
useState
}
// drop-in: const [theme, setTheme] = useLocalStorage<Theme>("theme", "dark");
```

- Returning `[value, setValue] as const` mirrors `useState`, so it's a one-line drop-in replacement. Hooks can call other hooks (composition).
- **Lazy initializer**: `useState(() => compute())` runs `compute` ONCE on first render (vs `useState(compute())` which runs every render).

Rules of Hooks (and why):

- **Only call hooks at the TOP LEVEL** — never in loops, conditions, nested functions, or after an early return. Why: React matches hooks to their stored state by CALL ORDER across renders; conditional hooks shift the order → wrong state. (The `use` prefix + ESLint enforce this.)
- **Only call hooks from React functions** (components or custom hooks).
- **Calling a hook ≠ using what it returned.** The rules govern `useX(...)` calls. Setters/ `dispatch` /values (`setCount`, `toggleTheme`, `count`) are ordinary — use them ANYWHERE, including event handlers. Updating state in a handler = calling a setter, not a hook.

Build & deploy (Lesson 15)

```
npm run build   # type-check + bundle → static files in dist/ (minified, hashed)
```

- `dist/` = your whole app as plain static files. Deploying = putting `dist/` on a host. No server/Node needed — that's why static hosting is free.
- **SPA deep-link gotcha:** visiting `/projects/x` directly 404s unless the host serves `index.html` for every path (then React Router takes over). Fix: `vercel.json` `rewrites` → `/index.html`, and/or `public/_redirects` (`/* /index.html 200`) for Netlify.
- **Deploy flow used:** `.gitignore` (ignore node_modules, dist) → `git init` → commit → push to GitHub → import on Vercel (auto-detects Vite, free Hobby tier). Every `git push` to main now auto-redeploys.
- **Git identity gotcha:** commits carry an author name+email. For a PUBLIC personal repo, set a repo-local personal identity (`git config user.email …` , a GitHub `…@users.noreply.github.com` address) so no work email leaks. The push CREDENTIAL (Windows Credential Manager) is separate from the commit identity — both must be the personal account.

TypeScript mini-glossary (concepts we flagged)

Thing	Means
`interface Foo { ... }`	a contract describing an object's shape
`string[]`	an array of strings; `Repo[]` = array of Repo
`field?: type`	optional property (may be missing)
`type T = "a" | "b"`	union — the value can only be one of these
`value: T | null`	union with null — must handle the null case

`useState<T>(...)`	generic: tells the hook what type it holds
`x!`	non-null assertion: "trust me, not null" (unchecked)
`??`	nullish coalescing: fallback when null/undefined
`as const`	treat as a fixed tuple/literal, not a loose array
`.d.ts`	types-only file (e.g. `vite-env.d.ts` for `.css` imports)
type narrowing	after `if (!x) return`, TS knows `x` isn't null below
`unknown`	"prove what this is before using it" — e.g. `catch (err)` is `unknown`; narrow with `err instanceof Error` before `.message`
discriminated union	union where each member shares a literal field (e.g. `type`); a `switch` on it narrows to the exact shape (reducer actions)

JavaScript syntax you hit along the way

- **Spread** `{ ...obj }` / `[ ...arr ]` — copy contents into a new object/array.
- **Computed key** `{ [variable]: value }` — the key comes from a variable.
- **Object shorthand** `{ theme }` = `{ theme: theme }`.
- **Destructuring** `const { name, value } = e.target` — pull fields into variables.
  - In a parameter, `{ x }: { x: number }` = destructure on the **left**, type on the **right** (split at the `:`).
- `.map()` — turn an array into an array of JSX (needs a `key`).
- `.reduce((acc, item) => ..., start)` — boil an array down to one value (e.g. a sum). The `start` value also prevents a crash on an empty array.
- **Promise /** `await` — a Promise is a future value; `await` waits for it. `async` functions always return a Promise.
- **Arrow returning an object** — wrap in parens: `() => ({ a: 1 })`.

CSS techniques (portfolio redesign)

```css
/* Gradient text — paint the letters with a gradient */
.name {
  background: linear-gradient(120deg, #fb7185, #a855f7, #38bdf8);
  -webkit-background-clip: text;
  background-clip: text;
  color: transparent;   /* hide the normal fill so the gradient shows through
*/
}
```

- **Layered backgrounds:** one element can stack many backgrounds, comma-separated (first listed = on top). Each layer can have its own `background-size` / `-position` / `-repeat` (also comma-separated, lined up by order).
- **Grid pattern** = two crossed gradients + a cell size: `linear-gradient(to right, var(--line) 1px, transparent 1px)` (vertical lines) `+` the `to bottom` version (horizontal), then `background-size: 44px 44px`.
- **Fade anything with a mask:** `mask-image: radial-gradient(#000 35%, transparent 80%)` — visible where the mask is opaque, hidden where transparent. Fades a grid/texture out at the edges. (Add the `-webkit-mask-image` twin for Safari.)
- **Radial glows:** `radial-gradient(ellipse 50% 55% at 80% 22%, color, transparent 60%)` — `ellipse W H` = size, `at X% Y%` = center. Stack two for depth/color.
- **Glow visibility = CONTRAST with the background.** Light mode needs higher opacity
  - bigger spread than dark for the *same* effect → keep separate light/dark tokens.
- **Keyframe animation** (two parts: define + apply): `@keyframes blink { 50% { opacity: 0 } }` then `animation: blink 1s step-end infinite`. `step-end` = hard on/off (terminal cursor); default easing = smooth pulse. Unspecified keyframes (0/100%) use the base value.
- **`background-position` % is ALIGNMENT, not distance:** `0%` aligns left edges, `100%` aligns right edges. With an OVERSIZED background, a higher % slides it LEFT (feels backwards). Pixels move intuitively; percentages don't. Swap values to flip direction.
- **Shine/shimmer sweep on hover:** an extra-wide white highlight layer parked off-screen; on `:hover` animate its `background-position` across, with a `transition`.
- **Respect motion sensitivity:** wrap looping/auto motion in `@media (prefers-reduced-motion: reduce) { animation: none; }`.
- **Theme-aware values:** define a CSS variable, override it under `:root[data-theme="light"]`, and reference the var everywhere (glows, header bg, grid lines) so one toggle reskins the whole page.
- **Outlined ("ghost") text:** `color: transparent` + `-webkit-text-stroke: 1.5px COLOR` = hollow letters. Fill on hover by setting `color` and making the stroke transparent. (Used for the big skill-card numbers.)
- **`color-mix()`** — two everyday uses:
  - *Alpha from a variable:* `color-mix(in srgb, var(--c) 25%, transparent)` = that color at 25% opacity. (You can't put a hex var inside `rgba()`, so this is how.)
  - *Blend two colors:* `color-mix(in srgb, var(--c) 22%, var(--border))` = 22% accent mixed into the border.
- **Per-element CSS variable:** set a var on each item (`.card:nth-child(1) { --card-accent: var(--brand-rose) }`) and reference it in the SHARED rules (`var(--card-accent)`). One set of rules, a different color per item. Vars inherit to children (so the number inside reads the card's `--card-accent`).
- **DRY color tokens:** hoist repeated brand colors into `:root` vars (`--brand-rose/purple/blue`) and reference them everywhere — change one, update the whole site.
- **`String(n).padStart(2, "0")`** (JS, not CSS) — pads a string to a length → `"1"` becomes `"01"` for tidy two-digit labels. `padEnd` is the other-side twin.

Last updated: Phase 2 (portfolio redesign) — hero + About + Skills. Covers Lessons 1–15 + CSS techniques. Companion to ROADMAP.md.